VOLLSTÄNDIGE REFERENZ · GEPRÜFT GEGEN VERSION 2.1.321

Claude Code Befehlsreferenz

Alle Terminal-Befehle, alle Optionen und alle Slash-Befehle innerhalb einer Sitzung — mit einer Erklärung, was sie tun.

Zusammengestellt von Christians virtuellem Server · 10.11.2026 · Aktualisierung der Fassung vom 8. November 2026 (2.1.319)

Wie diese Liste entstanden ist

Terminal-Befehle und Optionen (Teil A und B) sind direkt aus der installierten Fassung 2.1.321 ausgelesen — `claude --help` und die Hilfe jedes Unterbefehls.

Die Slash-Befehle (Teil C) stammen aus den Befehlsdefinitionen im Programm selbst. Das Ausleseverfahren erkennt auch Befehle, deren Beschreibung erst zur Laufzeit gebildet oder aus einer Variablen geholt wird, und seit dieser Fassung ebenso solche, deren *Name* in einer Variablen steht statt als Text im Programm. Beides kommt durch das Verdichten des Programmcodes beim Bauen zustande und sagt nichts über den Befehl selbst aus.

Befehle, die im Programm angelegt, aber per Schalter abgeschaltet sind, stehen gesondert am Ende, damit nichts als verfügbar erscheint, was es nicht ist.

Was sich seit der letzten Fassung geändert hat

Gegenüber 2.1.319: **ein neuer Unterbefehl, sonst nichts.** Die Hilfe von `claude` ist zeichengleich. In der Hilfe der Unterbefehle ist unter `claude plugin` neu: `restore-set-aside <plugin>` (gibt einer Plugin-Kennung die Optionen und Geheimnisse zurück, die beiseitegelegt wurden, als ein von Ihrer Organisation vorgegebener Marktplatz eine andere Quelle ersetzt hat).

Slash-Befehle: keine neuen, keine weggefallenen, und bei keinem hat sich die Beschreibung inhaltlich geändert.

Teil A · Terminal-Befehle und Optionen

Aufrufform: `claude [optionen] [befehl] [eingabe]`. Ohne Angaben startet eine Sitzung im aktuellen Ordner.

Sitzung starten

`claude`	Startet eine Sitzung im aktuellen Ordner.
`claude "deine Frage"`	Startet mit einer ersten Nachricht.
`claude -p "…"`	Antwortet einmal und beendet sich — für Skripte.

Optionen

`--add-dir <verzeichnisse...>`	Weitere Ordner, auf die Claude zugreifen darf.
`--agent <agent>`	Agent für diese Sitzung; überschreibt die Einstellung.
`--agents <json-oder-datei>`	Eigene Agenten als JSON mitgeben, mit --print auch als Pfad zu einer Datei.
`--allow-dangerously-skip-permissions`	Erlaubt das spätere Abschalten aller Rückfragen, ohne es sofort zu tun.

`--allowedTools, --allowed-tools <werkzeuge...>`	Nur diese Werkzeuge zulassen, z. B. "Bash(git *) Edit".
`--append-system-prompt <text>`	Text an die Systemanweisung anhängen.
`--autocompact <auto|tokens>`	Ab welcher Füllung automatisch zusammengefasst wird (auto oder 100k–1M).
`--ax-screen-reader`	Ausgabe für Screenreader: flacher Text, keine Rahmen, keine Animationen.
`--bg, --background`	Sitzung im Hintergrund starten und sofort zurückkehren. Gibt die Kennung aus, die claude attach, logs, stop und rm entgegennehmen; claude agents listet sie auf. Mit --resume läuft diese Sitzung unter derselben Kennung im Hintergrund weiter, oder es startet eine Kopie mit entsprechendem Hinweis, falls die Sitzung noch läuft.
`--bare`	Sparmodus: ohne Hooks, Plugins, Auto-Gedächtnis und CLAUDE.md-Suche.
`--betas <betas...>`	Beta-Kennungen an die Schnittstelle mitsenden (nur mit API-Schlüssel).
`--brief`	Erlaubt Claude, dir während der Arbeit kurze Zwischenmeldungen zu schicken.
`--chrome`	Claude-in-Chrome-Anbindung einschalten.
`--cloud [beschreibung|id|url]`	Cloud-Sitzung anlegen oder an eine bestehende andocken.
`-c, --continue`	Letzte Unterhaltung in diesem Ordner fortsetzen.
`--dangerously-skip-permissions`	Alle Rückfragen abschalten. Nur für abgeschottete Umgebungen ohne Internet.
`-d, --debug [filter]`	Fehlersuche einschalten, wahlweise gefiltert (z. B. --debug=hooks,mcp oder --debug='!hooks,!mcp').
`--debug-file <pfad>`	Fehlerprotokoll in diese Datei schreiben.
`--desktop`	Sitzung in der Claude-Desktop-App statt im Terminal öffnen (mit --continue oder --resume die Sitzung wählen).
`--disable-slash-commands`	Alle Skills und Befehle abschalten.
`--disallowedTools, --disallowed-tools <werkzeuge...>`	Diese Werkzeuge sperren.
`--effort <stufe>`	Denkaufwand für die Sitzung: low, medium, high, xhigh, max.
`--environment <id>`	Cloud-Sitzung in einer selbst betriebenen Umgebung starten.
`--exclude-dynamic-system-prompt-sections`	Rechnerabhängige Angaben aus der Systemanweisung heraushalten (bessere Zwischenspeicherung).
`--fallback-model <modell>`	Ersatzmodell, wenn das gewählte überlastet ist (auch außerhalb von --print).
`--file <angaben...>`	Dateien beim Start herunterladen (file_id:zielpfad).
`--fork-session`	Beim Fortsetzen eine neue Sitzungs-ID vergeben statt die alte weiterzuführen.
`--forward-subagent-text`	Text und Denkschritte der Unteragenten mit durchreichen.
`--from-pr [wert]`	Sitzung zu einem Pull Request fortsetzen (Nummer oder URL).
`-h, --help`	Hilfe anzeigen.
`--ide`	Beim Start automatisch mit der Entwicklungsumgebung verbinden.

`--include-hook-events`	Hook-Ereignisse mit in den Ausgabestrom schreiben (nur mit --print).
`--include-partial-messages`	Teilstücke der Antwort ausgeben, sobald sie eintreffen (nur mit --print).
`--input-format <format>`	Eingabeformat: text oder stream-json (nur mit --print).
`--json-schema <schema>`	JSON-Schema, gegen das die Antwort geprüft wird (nur mit --print).
`--max-budget-usd <betrag>`	Obergrenze in Dollar für die Schnittstellenaufrufe (nur mit --print).
`--mcp-config <dateien...>`	MCP-Server aus JSON-Dateien oder -Texten laden.
`--model <modell>`	Modell für diese Sitzung.
`-n, --name <name>`	Anzeigename der Sitzung.
`--no-chrome`	Chrome-Anbindung abschalten.
`--no-session-persistence`	Sitzung nicht auf die Festplatte schreiben.
`--output-format <format>`	Ausgabeformat: text, json oder stream-json (nur mit --print).
`--permission-mode <modus>`	Berechtigungsmodus: acceptEdits, plan, bypassPermissions, default.
`--permission-prompts <target>`	Wer bei --print auf Berechtigungsabfragen antwortet: "host" (SDK-Host bzw. --permission-prompt-tool) oder "none" (automatische Ablehnung; der Berechtigungsmodus entscheidet weiterhin über alles andere). Standard: host.
`--plugin-dir <pfad>`	Plugin aus einem Ordner oder Archiv laden, nur für diese Sitzung; ein Ordner mit mehreren Plugins lädt sie alle.
`--plugin-url <url>`	Plugin-Archiv von einer Adresse laden, nur für diese Sitzung.
`-p, --print`	Antwort ausgeben und beenden — für Skripte und Pipes.
`--prompt-suggestions [wert]`	Eingabevorschläge einschalten.
`--remote-control [name]`	Sitzung mit Fernsteuerung starten (Handy, claude.ai).
`--remote-control-session-name-prefix <prefix>`	Namensvorsatz für automatisch benannte Fernsteuer-Sitzungen.
`--replay-user-messages`	Eingelesene Nutzernachrichten zur Bestätigung zurückgeben.
`--restricted`	Eingeschränkter Modus: ohne Werkzeuge, die Befehle oder Code ausführen, ohne WebFetch und ohne Benutzer-, Projekt- und lokale Einstellungsdateien; Dateizugriff nur in den Arbeitsverzeichnissen.
`-r, --resume [wert]`	Unterhaltung per ID oder Name fortsetzen oder Auswahlliste öffnen.
`--safe-mode`	Ohne alle Anpassungen starten: keine CLAUDE.md, Skills, Plugins, Hooks.
`--session-id <uuid>`	Feste Sitzungs-ID vorgeben.
`--setting-sources <quellen>`	Welche Einstellungsquellen geladen werden: user, project, local.
`--settings <datei-oder-json>`	Zusätzliche Einstellungen aus Datei oder JSON-Text.
`--strict-mcp-config`	Nur die per --mcp-config angegebenen MCP-Server verwenden.
`--system-prompt <text>`	Systemanweisung für die Sitzung ersetzen.
`--system-prompt-snapshot <on|off>`	Systemanweisung einmal je Unterhaltung festhalten und danach bei jeder Anfrage sowie beim Fortsetzen unverändert wiederverwenden (empfohlen: on). Standardmäßig an (außer bei --bare außerhalb von Cloud- und Hintergrundsitzungen); ein --system-prompt oder --append-system-prompt wird mit festgehalten, auch wenn ein späterer Start anderen Text übergibt. off: bei jeder Anfrage frisch erzeugen, zum Feilen am Text.

`--teleport [sitzung]`	Teleport-Sitzung fortsetzen.
`--tmux`	tmux-Sitzung für den Arbeitsbaum anlegen (zusammen mit --worktree).
`--tools <werkzeuge...>`	Auswahl aus den eingebauten Werkzeugen festlegen.
`--verbose`	Ausführliche Ausgabe, unabhängig von der Einstellung.
`-v, --version`	Versionsnummer ausgeben.
`-w, --worktree [name]`	Neuen Git-Arbeitsbaum für diese Sitzung anlegen.

Teil B · Unterbefehle

Aufrufe der Form `claude <unterbefehl>`. Eingerückt die jeweils zugehörigen Unter-Unterbefehle.

`claude agents`	Hintergrund-Agenten verwalten.
`claude attach`	Eine Hintergrundsitzung in diesem Terminal öffnen.
`claude auth`	Anmeldung verwalten.
`claude auth login`	Am Anthropic-Konto anmelden.
`claude auth logout`	Abmelden.
`claude auth status`	Anmeldestatus anzeigen.
`claude auto-mode`	Einstellungen des automatischen Modus ansehen oder zurücksetzen.
`claude auto-mode config`	Wirksame Konfiguration als JSON ausgeben.
`claude auto-mode critique`	Rückmeldung zu eigenen Regeln einholen.
`claude auto-mode defaults`	Mitgelieferte Standardregeln ausgeben.
`claude auto-mode reset`	Auf die Standardregeln zurücksetzen.
`claude doctor`	Installation prüfen. Für die große Untersuchung /doctor in einer Sitzung.
`claude gateway`	Firmen-Gateway für Anmeldung und Telemetrie betreiben.
`claude import`	Konfiguration eines anderen KI-Werkzeugs übernehmen.
`claude install`	Native Fassung installieren (stable, latest oder feste Version).
`claude logs`	Letzte Terminalausgabe einer Hintergrundsitzung anzeigen.
`claude mcp`	MCP-Server einrichten und verwalten.
`claude mcp add`	MCP-Server hinzufügen (stdio, http oder sse).
`claude mcp add-from-claude-desktop`	Server aus Claude Desktop übernehmen (nur Mac und WSL).
`claude mcp add-json`	Server aus einem JSON-Text anlegen.
`claude mcp get`	Angaben zu einem Server anzeigen.
`claude mcp list`	Eingerichtete Server auflisten.
`claude mcp login`	An einem Server anmelden.

`claude mcp logout`	Von einem Server abmelden.
`claude mcp remove`	Server entfernen.
`claude mcp reset-project-choices`	Projektbezogene Zustimmungen zurücksetzen.
`claude mcp serve`	Claude Code selbst als MCP-Server betreiben.
`claude plugin`	Plugins verwalten.
`claude plugin configure`	Optionen eines Plugins anzeigen oder Werte von stdin speichern.
`claude plugin details`	Bestandteile eines Plugins und geschätzte Kontextkosten.
`claude plugin disable`	Plugin abschalten.
`claude plugin enable`	Plugin einschalten.
`claude plugin eval`	Testfälle gegen ein Plugin laufen lassen und bewerten.
`claude plugin init|new`	Gerüst für ein neues Plugin anlegen.
`claude plugin install|i`	Plugin aus einem Marktplatz installieren.
`claude plugin list`	Installierte Plugins auflisten.
`claude plugin marketplace`	Marktplätze verwalten.
`claude plugin prune|autoremove`	Nicht mehr benötigte Plugins entfernen.
`claude plugin restore-set-aside`	Beiseitegelegte Optionen und Geheimnisse eines Plugins wiederherstellen.
`claude plugin tag`	Git-Tag für eine Plugin-Veröffentlichung anlegen und dabei die Manifeste prüfen.
`claude plugin test`	Die Tests eines Mods ausführen.
`claude plugin uninstall|remove`	Plugin deinstallieren.
`claude plugin update`	Plugin aktualisieren.
`claude plugin validate`	Plugin auf Fehler prüfen.
`claude purge`	Die Claude-Code-Daten eines Projekts löschen (Transkripte, Aufgaben, Dateiverlauf, Eintrag in ~/.claude.json), nur auf diesem Rechner.
`claude respawn`	Hintergrundsitzung(en) neu starten, damit sie die aktuelle Fassung nutzen.
`claude rm`	Hintergrundsitzung löschen, auch ihren Arbeitsbaum, wenn das gefahrlos möglich ist.
`claude setup-token`	Langlebiges Anmeldetoken erzeugen (für Skripte und CI).
`claude stop|kill`	Hintergrundsitzung anhalten; die Unterhaltung bleibt erhalten.
`claude ultrareview`	Mehrstufige Code-Durchsicht in der Cloud starten.
`claude update|upgrade`	Auf Aktualisierungen prüfen und installieren; Installationen per Paketmanager aktualisieren sich über diesen.

Teil C · Slash-Befehle in der Sitzung

130 verfügbare Befehle, alphabetisch. Grau dahinter: Zweitnamen und erwartete Argumente.

`/add-dir` <path>	Weiteren Arbeitsordner hinzufügen.
`/advisor`	Claude an wichtigen Stellen ein stärkeres Modell befragen lassen.
`/agents`	Entfernt. Unteragenten liegen jetzt in .claude/agents/.
`/artifact-capabilities`	Was veröffentlichte Seiten zur Laufzeit können.
`/artifact-design`	Gestaltungsregeln für veröffentlichte Seiten.
`/artifact-diagramming`	Diagramme für veröffentlichte Seiten.
`/artifacts`	Veröffentlichte und geteilte Artefakte durchsehen.
`/auto-mode-setup` [--request-id <uuid>] (--wizard posture=\u2026 scope=\u2026 depth=\u2026 --propose | --expect-sha256 <64-hex> --apply-file <path>)	Automatischen Modus einrichten und anpassen.
`/autocompact` [auto|<tokens>]	Festlegen, ab welcher Füllung automatisch zusammengefasst wird.
`/autofix-pr`	Den aktuellen Pull Request überwachen und Probleme beheben.
`/background` auch /bg · [prompt]	Sitzung in den Hintergrund schicken und das Terminal freigeben.
`/batch` <instruction>	Große Änderung planen und dann parallel ausführen lassen.
`/branch` [name]	Die Unterhaltung an dieser Stelle verzweigen.
`/brief`	Knapp-Modus umschalten.
`/btw` [question]	Zwischenfrage stellen, ohne den laufenden Faden zu stören.
`/bug` auch /share · [report]	Fehler melden oder die Unterhaltung teilen.
`/chrome`	Einstellungen für Claude in Chrome öffnen.
`/claude-api`	Nachschlagewerk zur Claude-Schnittstelle (Modelle, Preise, Werkzeuge).
`/claude-code-docs` [question]	Fragen zu Claude Code selbst beantworten (Befehle, Optionen, Einstellungen, Hooks, Skills), geprüft gegen die installierte Fassung.
`/claude-in-chrome`	Browser fernsteuern: klicken, tippen, Seiten auslesen.
`/clear` auch /reset, /new · [name]	Neue Sitzung mit leerem Kontext; die alte bleibt erhalten.
`/cloud-plugins`	Festlegen, ob Cloud-Sitzungen die hier eingeschalteten Plugins mitbenutzen.
`/code-review` auch /review · […] [--fix] [--comment] [<pr#>|<branch>|<path>] [--max-findings <n>|all]	Änderungen auf Fehler und Vereinfachungen durchsehen.
`/color`	Farbe der Eingabezeile für diese Sitzung.
`/commit` [guidance]	Legt einen Git-Commit an, mit passend geschriebener Nachricht.
`/commit-push-pr`	Committen, hochladen und einen Pull Request eröffnen – in einem Zug.
`/compact` <optional custom summarization instructions>	Kontext freimachen, indem das Bisherige zusammengefasst wird.
`/config` auch /settings · [key=value]	Einstellungen öffnen.
`/context` [all]	Kontextverbrauch als farbiges Raster anzeigen.

`/copy`	Letzte Antwort in die Zwischenablage kopieren (/copy N für ältere).
`/cowork-plugin`	Legt ein neues Cowork-Plugin an oder passt ein vorhandenes an.
`/daemon`	Hintergrunddienste und Routinen verwalten.
`/dataviz`	Diagramme und Auswertungsseiten sauber gestalten.
`/debug` [issue description]	Fehlersuche für diese Sitzung einschalten.
`/design` [sync|login|consent|revoke|import|export|status|<prompt>]	Zugriff auf Design-Projekte erteilen oder entziehen.
`/design-consent`	Zugriff auf Design-Projekte erteilen.
`/design-login`	Design-Zugriff für /design-sync freischalten.
`/design-revoke`	Zugriff auf Design-Projekte entziehen.
`/design-sync`	Ein React-Designsystem nach claude.ai/design übertragen.
`/desktop` auch /app	Die laufende Sitzung in der Desktop-App weiterführen.
`/diff`	Nicht eingecheckte Änderungen und Änderungen je Zug ansehen.
`/doctor` auch /checkup · [prompt-audit [<path>]]	Installation durchleuchten und Probleme beheben.
`/effort`	Denkaufwand einstellen.
`/exit` auch /quit	Sitzung beenden.
`/explain-usage`	Den eigenen Verbrauch erklären lassen.
`/export` [filename]	Unterhaltung in eine Datei oder die Zwischenablage ausgeben.
`/extra-usage`	Umbenannt in /usage-credits.
`/fast` [on|off]	Schnellmodus umschalten (gleiches Modell, höheres Tempo).
`/feedback` [report]	Rückmeldung an Anthropic senden.
`/fewer-permission-prompts`	Häufige harmlose Aufrufe erkennen und dauerhaft erlauben.
`/focus` [on|off]	Nur Eingabe, Zusammenfassung und Antwort anzeigen.
`/fork` <directive>	Hintergrund-Agent starten, der die ganze Unterhaltung erbt.
`/goal` [<condition> | clear]	Ein Ziel setzen, das Claude vor dem Aufhören prüft.
`/heapdump`	Speicherabbild schreiben (zur Fehlersuche).
`/help`	Hilfe und verfügbare Befehle anzeigen.
`/hooks`	Eingerichtete Hooks für Werkzeug-Ereignisse ansehen.
`/ide` [open]	Anbindung an die Entwicklungsumgebung verwalten.
`/import` [codex|gemini|cursor] [--dry-run]	Konfiguration eines anderen KI-Werkzeugs übernehmen.
`/init`	Eine erste CLAUDE.md aus dem vorhandenen Code erzeugen.
`/insights`	Auswertung der eigenen Sitzungen erstellen.
`/install` [options]	Native Fassung installieren.
`/install-github-app`	Claude-GitHub-Aktionen für ein Repository einrichten.
`/install-slack-app`	Claude-App für Slack installieren.
`/keybindings`	Datei mit den Tastenkürzeln öffnen.

`/limit-reset`	Das 5-Stunden-Limit sofort zurücksetzen und weiterarbeiten; einmal pro Woche möglich, zählt weiter aufs Wochenlimit.
`/list-agents` auch /peers	Unteragenten und andere erreichbare Sitzungen auflisten.
`/login`	Am Anthropic-Konto anmelden oder Konto wechseln.
`/logout`	Vom Anthropic-Konto abmelden.
`/loop` auch /proactive · [interval] [prompt]	Einen Auftrag in festem Abstand wiederholt laufen lassen.
`/low-priority`	Nach Erreichen des 5-Stunden-Limits mit niedrigerer Priorität fortsetzen; erneuter Aufruf stoppt das wieder.
`/mcp` [reconnect (<server>|all)|enable|disable [<server>|all]]	MCP-Server verwalten.
`/memory`	CLAUDE.md-Dateien und Gedächtnis-Einstellungen bearbeiten.
`/mobile` auch /ios, /android	QR-Code für die Handy-App anzeigen.
`/model` [model]	Modell für Claude Code einstellen.
`/output-style` [style]	Ausgabestile auflisten oder zu einem wechseln.
`/passes`	Eine Gratiswoche Claude Code verschenken.
`/permissions` auch /allowed-tools	Regeln für erlaubte und verbotene Werkzeuge verwalten.
`/plan` [open|<description>]	Plan-Modus einschalten oder den Plan der Sitzung ansehen.
`/plugin` auch /plugins, /marketplace	Plugins verwalten.
`/plugin-authoring`	Hilft beim Schreiben und Prüfen eines Plugins aus Funktions-Hooks.
`/powerup`	Funktionen von Claude Code in kurzen Lektionen kennenlernen.
`/privacy-settings`	Datenschutz-Einstellungen ansehen und ändern.
`/pro-trial-expired`	Auswahl nach Ablauf der Pro-Testphase.
`/project` [Project ID or claude.ai address] [--save] | clear [--save]	Das claude.ai-Projekt festlegen, dem die Nutzungsberichte dieser Sitzung zugeordnet werden.
`/radio`	Lo-Fi-Radio einschalten.
`/rate-limit-options`	Möglichkeiten anzeigen, wenn das Nutzungslimit erreicht ist.
`/recap`	Einzeilige Zusammenfassung der Sitzung erzeugen.
`/release-notes`	Neuerungen der Versionen anzeigen.
`/reload-plugins` [--force]	Ausstehende Plugin-Änderungen übernehmen.
`/reload-skills`	Neu hinzugekommene oder geänderte Skills einlesen.
`/remote-control` auch /rc	Sitzung vom Handy oder von claude.ai aus steuern.
`/remote-env`	Standardumgebung für Cloud-Agenten wählen.
`/rename` auch /name · [name]	Die Unterhaltung umbenennen.
`/restart` auch /update	Claude Code neu starten und diese Sitzung behalten.
`/resume` auch /continue	Frühere Unterhaltung fortsetzen.
`/rewind` auch /checkpoint, /undo	Code und/oder Unterhaltung auf einen früheren Stand zurücksetzen.
`/run`	Die Anwendung starten und eine Änderung im Betrieb prüfen.
`/run-skill-generator`	Aus einem Ablauf einen neuen Skill erzeugen lassen.

`/sandbox`	Abschottung einrichten und anzeigen.
`/schedule` auch /routines	Geplante Cloud-Agenten anlegen und verwalten.
`/scroll-speed`	Rollgeschwindigkeit des Mausrads einstellen.
`/session` auch /remote	Adresse und QR-Code der Cloud-Sitzung anzeigen.
`/setup-bedrock`	Amazon Bedrock neu einrichten.
`/setup-claude` auch /setup-cowork	Führt durch die Einrichtung: Rolle wählen, passendes Plugin installieren, Skill ausprobieren, Werkzeuge verbinden.
`/setup-vertex`	Google Vertex AI neu einrichten.
`/simplify` [<target>]	Geänderten Code aufräumen und vereinfachen.
`/skill-doctor`	Zeigen, welche geladenen Skills ungenutzt Kontext kosten.
`/skills`	Verfügbare Skills auflisten.
`/status`	Zustand anzeigen: Version, Modell, Konto, Anbindung.
`/statusline`	Statuszeile einrichten.
`/stickers`	Claude-Code-Aufkleber bestellen.
`/stop`	Diese Hintergrundsitzung beenden; Protokoll und Arbeitsbaum bleiben.
`/subtask` <task>	Unteragent mit vollem Kontext losschicken; Ergebnis kommt zurück.
`/tasks` auch /bashes	Alles ansehen und verwalten, was im Hintergrund läuft.
`/team-onboarding`	Anleitung für Kollegen aus der eigenen Nutzung erzeugen.
`/teleport` auch /tp	Eine Sitzung von claude.ai hier fortsetzen.
`/terminal-setup`	Tastenbelegung des Terminals einrichten (Zeilenumbruch).
`/theme`	Farbschema wechseln.
`/tui` [default|fullscreen]	Oberflächenart wählen: normal oder bildschirmfüllend.
`/ultraplan` <prompt>	Einen bearbeitbaren Plan im Web entwerfen lassen.
`/ultrareview`	Mehrstufige Fehlersuche im Zweig, in der Cloud.
`/upgrade`	Den Tarif wechseln für höhere Grenzen und mehr Opus.
`/usage` auch /cost, /stats	Kosten der Sitzung, Verbrauch und Aktivität anzeigen.
`/usage-credits`	Guthaben einrichten oder beim Administrator anfragen.
`/use-case` [use case]	Den Life-Sciences-Anwendungsfall wählen, der zu den eigenen Anfragen passt.
`/verify`	Prüft nach, ob eine Änderung wirklich tut, was sie soll – im echten Durchlauf.
`/voice` [hold|tap|off]	Sprachmodus umschalten.
`/web-setup`	Claude Code im Web mit dem GitHub-Konto verbinden.
`/workflow-launch-exec`	Vom Server angestoßenen Arbeitsablauf ausführen.
`/workflows`	Laufende und abgeschlossene Arbeitsabläufe durchsehen.

Anhang · Angelegt, aber abgeschaltet

Diese Namen existieren im Programm, sind in dieser Fassung aber per Schalter deaktiviert und stehen in einer Sitzung nicht zur Verfügung.

`/loops`	Wiederholungen auflisten, anlegen und löschen. im Programm angelegt, aber abgeschaltet
`/pause-memory` auch /memory-pause, /toggle-memory	Konto-Gedächtnis und lokales Auto-Gedächtnis für diese Sitzung anhalten (erneut aufrufen schaltet beide wieder ein). im Programm angelegt, aber abgeschaltet
`/version`	Version dieser Sitzung anzeigen. im Programm angelegt, aber abgeschaltet
`/wellbeing` auch /breaks, /break-reminder, /downtime	Pausenerinnerungen und Ruhezeiten einstellen. im Programm angelegt, aber abgeschaltet

---

Claude Code 2.1.321 · 66 Optionen · 130 verfügbare Slash-Befehle · 4 abgeschaltet · Stand 10.11.2026